Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «Обработка изображений в ИС»
Тема: «Трекинг множественных объектов»

Выполнил:
Студент 4 курса
Группы ИИ-26
Шоева Е.Т.
Проверила:
Андренко К.В.

Брест 2026

Цель: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов

Ход работы

Общее задание

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;
2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);
3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;
4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

tracking.py

```
import argparse
import csv
import importlib.util
import time
from pathlib import Path


import cv2
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import yaml

EXPERIMENTS = [
    ("bytetrack_default", "bytetrack", {}),
    ("botsort_default", "botsort", {}),
    ("bytetrack_buffer15", "bytetrack", {"track_buffer": 15}),
    ("bytetrack_buffer60", "bytetrack", {"track_buffer": 60}),
    ("bytetrack_buffer120", "bytetrack", {"track_buffer": 120}),
    ("bytetrack_high0.5", "bytetrack", {"track_high_thresh": 0.5, "new_track_thresh": 0.5}),
    ("bytetrack_match0.6", "bytetrack", {"match_thresh": 0.6}),
    ("bytetrack_match0.95", "bytetrack", {"match_thresh": 0.95}),
    ("bytetrack_high0.15", "bytetrack", {"track_high_thresh": 0.15, "new_track_thresh": 0.15}),
    ("botsort_high0.15", "botsort", {"track_high_thresh": 0.15, "new_track_thresh": 0.15}),
    ("botsort_buffer60", "botsort", {"track_buffer": 60}),
    ("botsort_gmc_none", "botsort", {"gmc_method": "none"}),
    ("botsort_prox0.3", "botsort", {"proximity_thresh": 0.3}),
    ("botsort_prox0.7", "botsort", {"proximity_thresh": 0.7}),
    ("botsort_reid", "botsort", {"with_reid": True}),
]



def base_yaml(name):
    spec = importlib.util.find_spec("ultralytics")
    root = Path(list(spec.submodule_search_locations)[0])
    return root / "cfg" / "trackers" / f"{name}.yaml"



def make_config(base_path, overrides, out_path):
    cfg = yaml.safe_load(Path(base_path).read_text(encoding="utf-8"))
    for k, v in overrides.items():
        cfg[k] = v
    Path(out_path).write_text(yaml.safe_dump(cfg, sort_keys=False), encoding="utf-8")
    return cfg



def compute_metrics(frames_ids, short_len=15, gt_objects=None):
    tracks = {}
    for f, ids in enumerate(frames_ids):
        for i in ids:
```

```
            tracks.setdefault(i, []).append(f)
    lengths = [len(v) for v in tracks.values()]
    gaps = 0
    for v in tracks.values():
        gaps += sum(1 for a, b in zip(v, v[1:]) if b - a > 1)
    n_frames = len(frames_ids)
    per_frame = [len(x) for x in frames_ids]
    m = {
        "frames": n_frames,
        "unique_ids": len(tracks),
        "max_concurrent": max(per_frame) if per_frame else 0,
        "avg_per_frame": round(sum(per_frame) / n_frames, 2) if n_frames else 0,
        "mean_track_len": round(sum(lengths) / len(lengths), 1) if lengths else 0,
        "median_track_len": sorted(lengths)[len(lengths) // 2] if lengths else 0,
        "short_tracks": sum(1 for x in lengths if x < short_len),
        "short_share": round(sum(1 for x in lengths if x < short_len) / len(lengths), 3) if lengths
else 0,
        "gaps": gaps,
        "frames_without_objects": sum(1 for x in per_frame if x == 0),
    }
    if gt_objects:
        m["gt_objects"] = gt_objects
        m["id_overcount"] = len(tracks) - gt_objects
    return m, tracks



def plot_timeline(tracks, n_frames, title, out_path):
    fig, ax = plt.subplots(figsize=(10, max(2.5, 0.25 * len(tracks) + 1)))
    for row, (tid, fr) in enumerate(sorted(tracks.items(), key=lambda kv: kv[1][0])):
        ax.plot(fr, [row] * len(fr), ".", markersize=2)
        ax.text(fr[0], row + 0.25, str(tid), fontsize=6)
    ax.set_xlim(0, max(n_frames, 1))
    ax.set_xlabel("кадр")
    ax.set_ylabel("трек")
    ax.set_title(title)
    plt.tight_layout()
    fig.savefig(out_path, dpi=130)
    plt.close(fig)



def run_experiment(name, weights, video, tracker_yaml, conf, iou, imgsz, classes, out_dir, device,
gt_objects, save_video):
    from ultralytics import YOLO

    model = YOLO(weights)
    cap = cv2.VideoCapture(str(video))
    fps = cap.get(cv2.CAP_PROP_FPS) or 25
    w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    cap.release()
    writer = None
    if save_video:
        writer = cv2.VideoWriter(str(Path(out_dir) / f"{name}.mp4"),
cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))
    frames_ids = []
    t0 = time.time()
    stream = model.track(source=str(video), tracker=str(tracker_yaml), conf=conf, iou=iou,
imgsz=imgsz,
                         classes=classes, device=device, stream=True, persist=True, verbose=False)
    for r in stream:
        ids = [] if r.boxes.id is None else [int(x) for x in r.boxes.id.tolist()]
        frames_ids.append(ids)
        if writer is not None:
            writer.write(r.plot())
    elapsed = time.time() - t0
    if writer is not None:
        writer.release()
    m, tracks = compute_metrics(frames_ids, gt_objects=gt_objects)
    m["fps"] = round(len(frames_ids) / elapsed, 1) if elapsed > 0 else 0
    plot_timeline(tracks, len(frames_ids), name, Path(out_dir) / f"{name}_timeline.png")
    return m
```

```
def save_summary(rows, out_dir):
    if not rows:
        return
    keys = list(rows[0].keys())
    for r in rows:
        for k in r:
            if k not in keys:
                keys.append(k)
    with open(Path(out_dir) / "summary.csv", "w", newline="", encoding="utf-8-sig") as f:
        wr = csv.DictWriter(f, fieldnames=keys)
        wr.writeheader()
        wr.writerows(rows)
    names = [r["experiment"] for r in rows]
    fig, axs = plt.subplots(1, 3, figsize=(15, 5))
    for ax, key, title in zip(axs, ["unique_ids", "mean_track_len", "fps"], ["Уникальных ID",
"Средняя длина трека, кадров", "Скорость, кадр/с"]):
        ax.barh(names, [r[key] for r in rows])
        ax.set_title(title)
        ax.invert_yaxis()
    plt.tight_layout()
    fig.savefig(Path(out_dir) / "summary.png", dpi=130)
    plt.close(fig)



def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--weights", required=True)
    ap.add_argument("--video", required=True)
    ap.add_argument("--out", default="tracking_results")
    ap.add_argument("--conf", type=float, default=0.25)
    ap.add_argument("--iou", type=float, default=0.7)
    ap.add_argument("--imgsz", type=int, default=640)
    ap.add_argument("--classes", type=int, nargs="*", default=[10])
    ap.add_argument("--device", default="0")
    ap.add_argument("--gt-objects", type=int, default=None)
    ap.add_argument("--only", nargs="*", default=None)
    ap.add_argument("--no-video", action="store_true")
    ap.add_argument("--list", action="store_true")
    a = ap.parse_args()

    if a.list:
        for n, base, ov in EXPERIMENTS:
            print(n, base, ov)
        return

    out = Path(a.out) / Path(a.video).stem
    cfg_dir = out / "configs"
    cfg_dir.mkdir(parents=True, exist_ok=True)
    rows = []
    for name, base, overrides in EXPERIMENTS:
        if a.only and name not in a.only:
            continue
        cfg_path = cfg_dir / f"{name}.yaml"
        make_config(base_yaml(base), overrides, cfg_path)
        print(f"=== {name} {overrides}")
        try:
            m = run_experiment(name, a.weights, a.video, cfg_path, a.conf, a.iou, a.imgsz,
a.classes,
                               out, a.device, a.gt_objects, not a.no_video)
        except Exception as e:
            print(f"ошибка в {name}: {e}")
            continue
        row = {"experiment": name, "tracker": base, "params": str(overrides)}
        row.update(m)
        rows.append(row)
        print(m)
        save_summary(rows, out)
    print(f"Готово. Результаты: {out}")



if __name__ == "__main__":
    main()
```



Результат детекции на кадрах видео (conf = 0,05, imgsz = 1280)

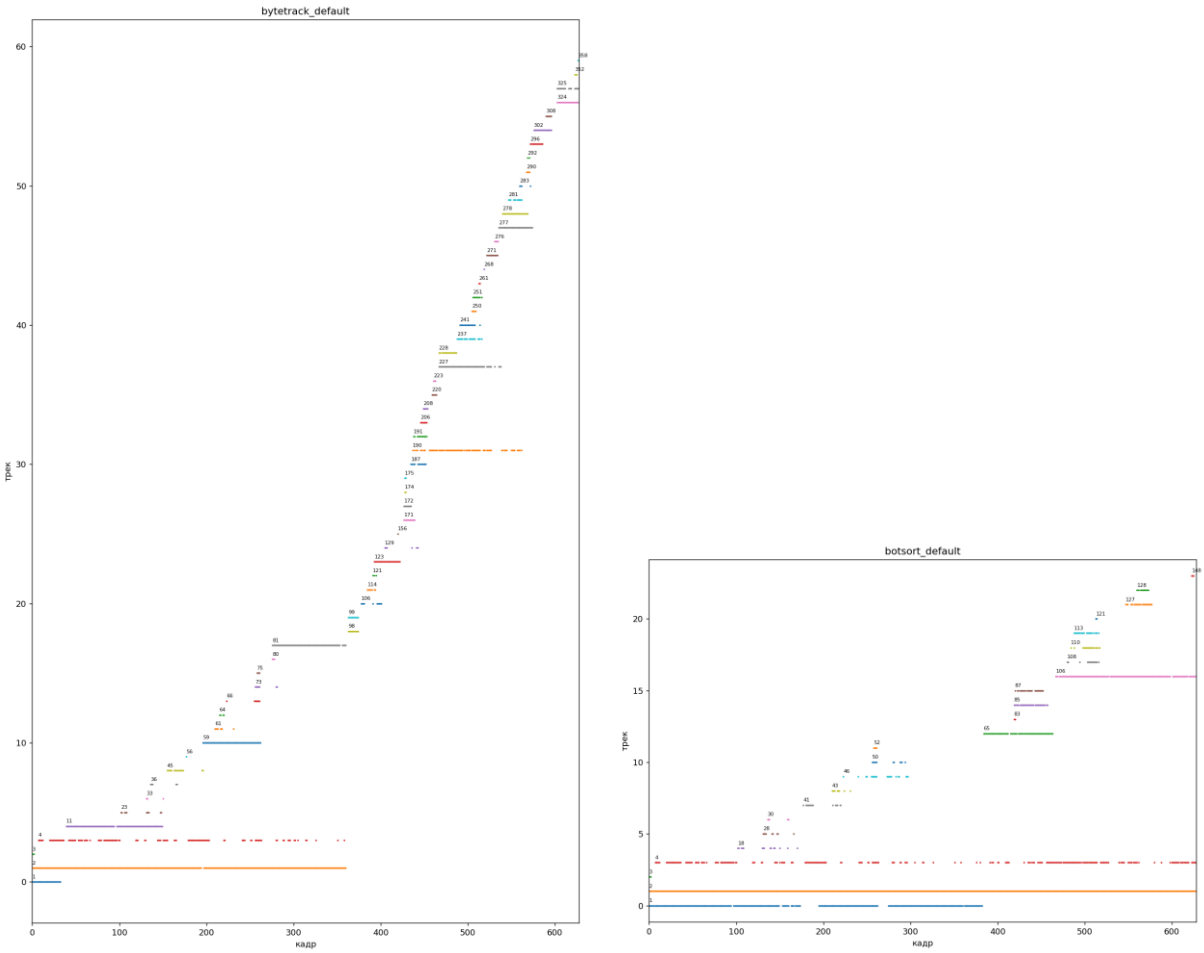
Сравнение алгоритмов с параметрами по умолчанию. В таблице приведены результаты ByteTrack и BoT-SORT на одном и том же видео и с одинаковыми настройками детектора.

Сравнение ByteTrack и BoT-SORT (параметры по умолчанию)

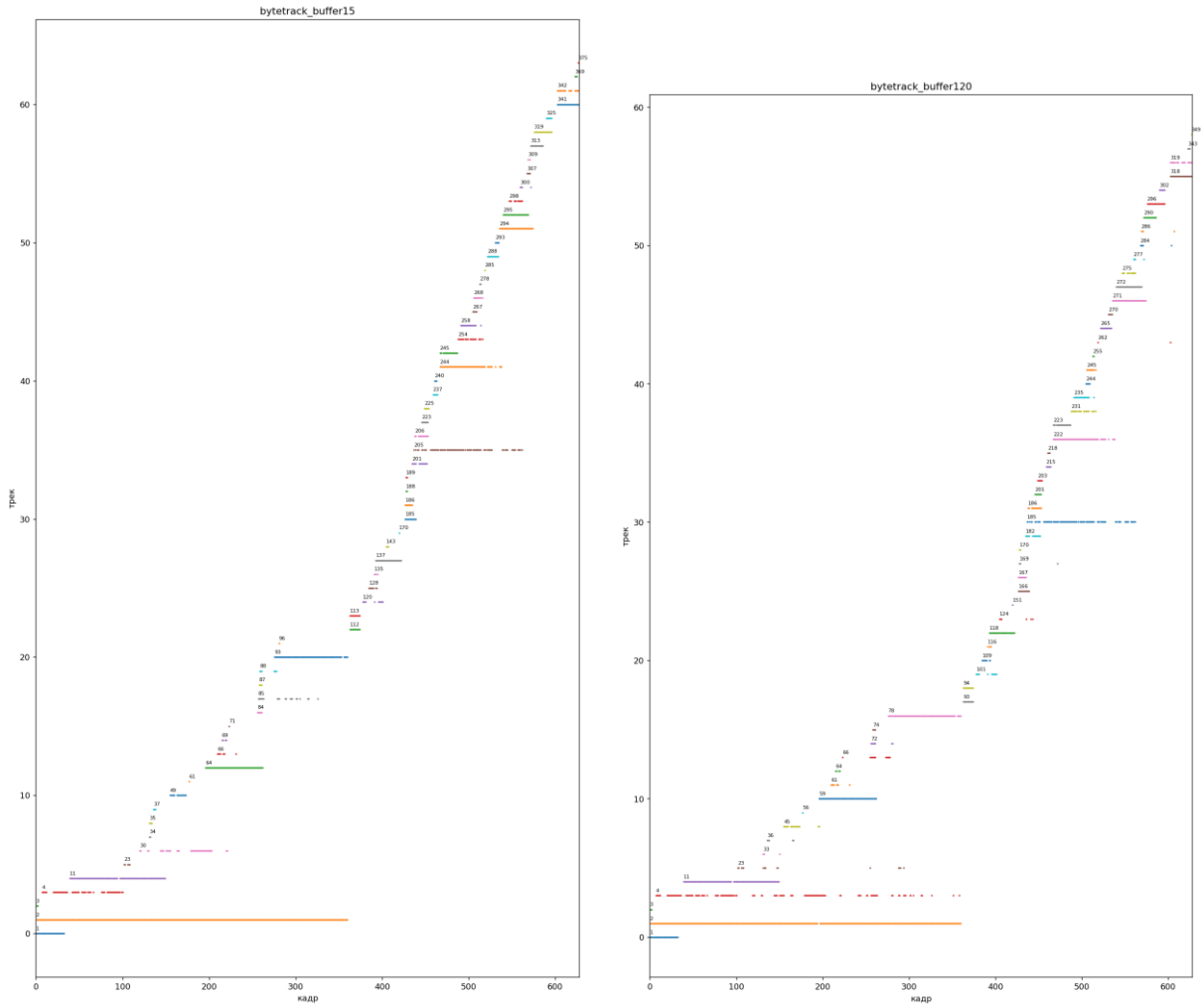
Метрика	**ByteTrack**	**BoT-SORT**
Кадров	628	628
Уникальных ID	60	24
Макс. одновременно объектов	6	7
Среднее объектов на кадр	2,31	2,75
Средняя длина трека, кадров	24,2	72
Медианная длина трека, кадров	9	16
Коротких треков (<15 кадров)	40 (66,7 %)	10 (41,7 %)
Разрывов треков (gaps)	88	126
Кадров без объектов	23	0
Скорость, кадр/с	8,4	8,1



Кадры из результата ByteTrack



Диаграммы жизни треков: а) ByteTrack, б) BoT-SORT (параметры по умолчанию)

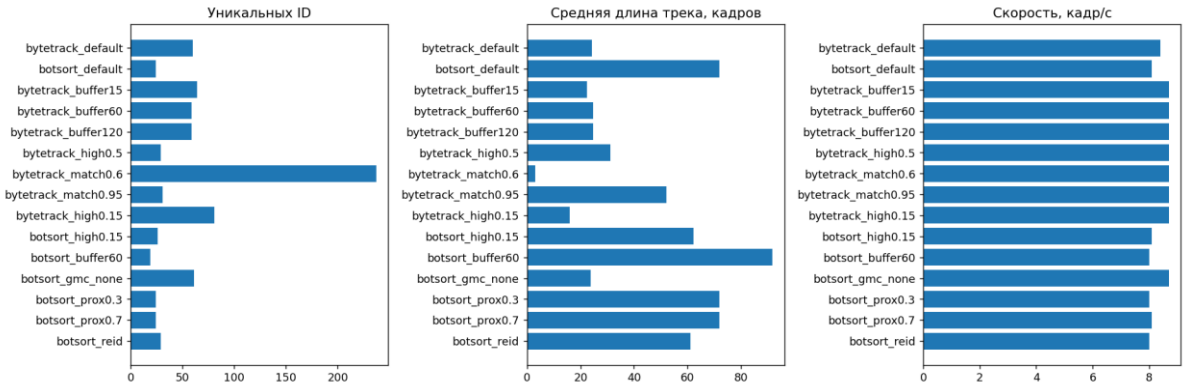


Диаграммы жизни треков ByteTrack: а) track_buffer = 15, б) track_buffer = 120

Влияние параметров конфигурации. Каждый эксперимент отличается от базового для своего алгоритма изменением одного параметра либо пары связанных порогов (таблица далее). Сводные графики приведены на рисунке далее.

Результаты экспериментов с параметрами трекеров

Эксперимент	**Изменение**	**ID**	**Объектов на кадр**	**Средняя длина**	**Медиана**	**Коротких, %**	**Gaps**	**FPS**
bytetrack_default	по умолчанию	60	2,31	24,2	9	66,7	88	8,4
botsort_default	по умолчанию	24	2,75	72	16	41,7	126	8,1
bytetrack_buffer15	track_buffer = 15	64	2,28	22,4	8	65,6	72	8,7
bytetrack_buffer60	track_buffer = 60	59	2,32	24,7	10	66,1	92	8,7
bytetrack_buffer120	track_buffer = 120	59	2,33	24,8	10	64,4	96	8,7
bytetrack_high0.5	high = new = 0,5	29	1,44	31,1	13	58,6	25	8,7
bytetrack_high0.15	high = new = 0,15	81	2,07	16	6	80,2	139	8,7
bytetrack_match0.6	match_thresh = 0,6	237	1,16	3,1	1	96,6	52	8,7
bytetrack_match0.95	match_thresh = 0,95	31	2,58	52,2	12	54,8	118	8,7
botsort_buffer60	track_buffer = 60	19	2,77	91,7	29	26,3	138	8
botsort_high0.15	high = new = 0,15	26	2,57	62,2	14	53,8	234	8,1
botsort_gmc_none	gmc_method = none	61	2,31	23,8	8	68,9	82	8,7
botsort_prox0.3	proximity_thresh = 0,3	24	2,75	72	16	41,7	126	8
botsort_prox0.7	proximity_thresh = 0,7	24	2,75	72	16	41,7	126	8,1
botsort_reid	with_reid = true	29	2,82	61	15	48,3	134	8



Сводные результаты

Вывод: исследовала применение алгоритмов трекинга на базе обученной сети-детектора объектов